

Evolutionary Algorithms: Final report

Matties Roofthoof (r0877229)

December 27, 2025

1 Metadata

- **Group members during group phase:** Kaixi Yao, Sarthak Janjuha and Julius Jacobitz
- **Time spent on group phase:** 10 hours
- **Time spent on final code:** 65 hours
- **Time spent on final report:** 20 hours

2 Changes since the group phase

(target: 0.25 pages)

1. Replaced the representation from N-1 cities to N cities always starting at city zero.
2. Replaced random initialization with a **greedy initialization** on a **noisy distance matrix**
3. Use **OX**, **ERX** and **EPX** as a crossoveroperator with different probability
4. Added local search operators: **symmetric 2-opt**, **symmetric 3-opt** and **Or-opt**.
5. Added diversity promotion: **islands** and **fitness sharing**
6. Adaptively adjust diversity promotion and local search

3 Final design of the evolutionary algorithm

(target: 3.5 pages)

3.1 The three main features

1. Greedy initialization with noisy distance matrix, discussed in section 3.4.
2. Local search operator: **Or-opt**, discussed in section 3.9.
3. Island specialization, discussed in section 3.7.

3.2 The main loop

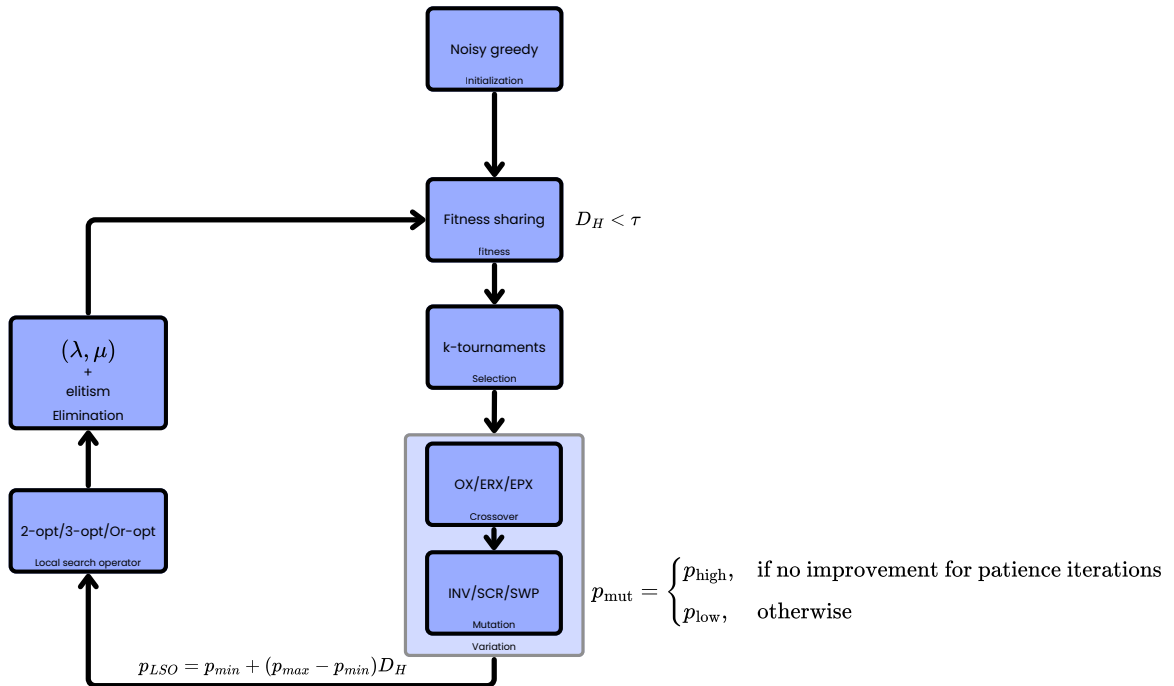


Figure 1: Flow genetic algorithm with D_H as the Hamming diversity on the Island

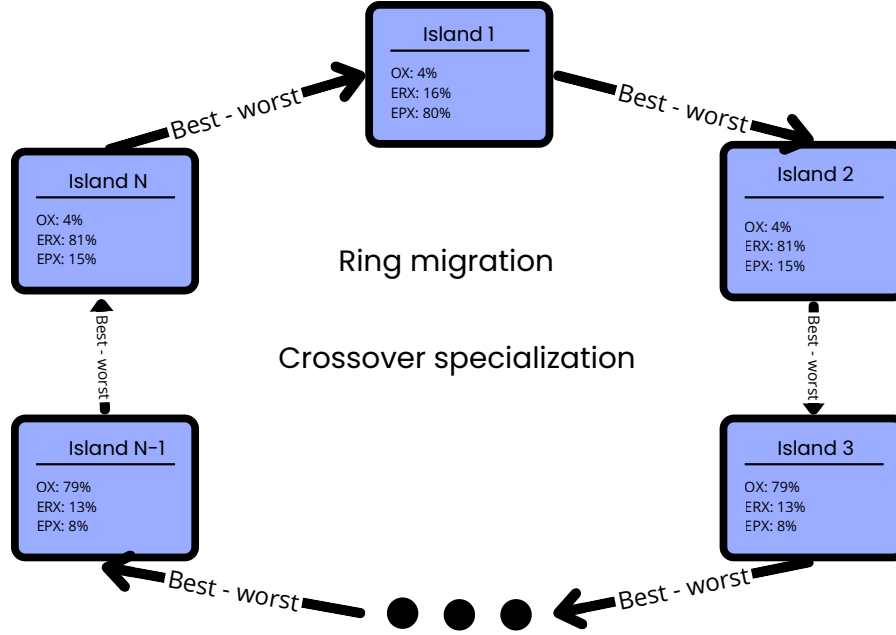


Figure 2: Island model diagram

3.3 Representation

A candidate solution is represented as a list of cities visited in order. This representation is intuitive and vectorizable if implemented as an array in `numpy`. We make sure that every candidate solution starts with city zero. This allows us to use **Hamming Distance** as a metric for population diversity. Another way to represent a candidate solution would be to use a binary matrix which indicates which city is visited next. However, for large matrices this will require a lot of memory $O(N^2)$.

In python the representation is implemented as a **numpy array**. This choice is preferred due to the compability with **numba** which significantly accelerates computation time.

3.4 Initialization

The population in each island is intialized using a **greedy heuristic** on a **noisy distance matrix**. The greedy heuristic allows us to find initial solutions which are better than purely random solutions. Especially for large matrices this initial leap in performance is crucial since purely **random initialization** will not be able to converge in the allowed time. If the distance matrix contains unconnected cities, the greedy approach allows us to find possible tours faster. We preserve diversity by adding noise on the distance matrix. By varying the amount of noise on this matrix we can control the tradeoff between diversity of the initial population and initial gain.

BFS and **DFS initialization** were implemented as well with the idea of finding random initialization which are feasible when the graph is not fully connected. But were left out due to their computational cost and the fact that for large matrices the initial bests were too high to be able to converge leading to limited benefit.

The population size is chosen based on reasoned trial and error. A larger population promotes exploration of the search space, whereas a smaller population emphasizes exploitation of known good solutions.

3.5 Selection operators

The implemented selection operator is **k-tournaments** since we have direct control on the selection pressure and it works well even when fitness values have a large range (like our tour lengths), as it only compares relative fitness within the tournament group. Small k (e.g., 2-3) means a more diverse selection, weaker individuals have a better chance. Large k (e.g., 5-10) means a more aggressive selection, strongly favoring the best individuals.

3.6 Mutation operators

We implemented **inversion**, **scramble** and **swap mutations**. Inversion and scramble mutation are **global mutations** and the segment length is randomly picked between zero and the number of cities. Swap mutation is a

local mutation. The mutation rate is **adaptively increased** to a fixed rate whenever the number of iterations without improvement in an island exceeds a patience parameter. Whenever an improvement is found, the mutation rate drops back to the base rate.

3.7 Recombination operators

We implemented three recombination operators: **Ordered Crossover (OX)**, **Edge-Preserving Crossover (EPX)**, and **Edge Recombination Crossover (ERX)**. OX copies a subsequence from one parent and fills the remaining cities in the order of the second parent, preserving relative city order. EPX and ERX focus on preserving edges from both parents, with ERX building an edge map to guide offspring construction. These operators were chosen because they produce valid tours while retaining promising building blocks, such as good subsequences or edges, increasing the likelihood of high-quality offspring. When parents share few edges, edge-based operators naturally produce more exploratory offspring. Operator selection is probabilistic, and while we currently use fixed probabilities, this could be extended to self-adaptive schemes.

Simpler operators like **PMX** or **Cycle Crossover** were considered but discarded due to poor edge preservation, and recombination is restricted to two parents for simplicity.

To further promote diversity, each island uses a different **mix of probabilities** for these **three operators**, allowing islands to specialize in certain crossovers and explore complementary regions of the search space. This diversity in crossover can be controlled by two hyperparameters. The first one controls how **diverse** probabilities are compared to a base distribution, the second one determines how much **islands specialize** in one particular crossover.

3.8 Elimination operators

We implemented (λ, μ) elimination with **elitism** for our genetic algorithm. Initially, we tested the $(\lambda + \mu)$ scheme, which preserves all elites automatically. However, we observed that this approach led to rapid loss of population diversity, as the best individuals dominated too quickly. To maintain exploration while still retaining the top solutions, we switched to (λ, μ) elimination combined with elitism, which explicitly preserves a small number of the best individuals each generation. This approach allows the majority of the population to be replaced, promoting diversity, while ensuring that the best solutions are never lost. No advanced parameter control (e.g., self-adaptivity) was applied to the elitism fraction.

3.9 Local search operators

We implemented **symmetric 2-opt** and **3-opt** local search operators, which rely on efficient **delta evaluations** to assess whether local permutations yield an improvement in tour cost. These operators are only valid for symmetric adjacency matrices, as applying them to asymmetric instances may result in incorrect improvements. The use of delta computations significantly reduces the computational cost compared to full tour evaluations. In addition, we implemented **Or-opt**, which can be viewed as a restricted variant of 3-opt. Or-opt relocates contiguous segments without reversing their orientation and is therefore applicable to problems with **asymmetric adjacency matrices**. Segment-swap operators were also considered to further increase population diversity. However, their empirical benefit was limited, and they were therefore omitted from the final algorithm.

The probability of applying a local search operator, denoted by p_{LSO} , is **adaptively adjusted** based on the population diversity, which is measured using the normalized **Hamming distance** D_H . The probability is interpolated between predefined minimum and maximum values as D_H varies between 0 and 1. When population diversity is high, a larger p_{LSO} promotes intensive exploitation of promising solutions. Conversely, as diversity decreases, the probability is reduced to limit excessive exploitation and prevent premature convergence to a single solution.

3.10 Diversity promotion mechanisms

To promote diversity and prevent premature convergence, we employ a combination of an **island model** and **fitness sharing**. The island model partitions the population into multiple semi-isolated subpopulations (islands), each evolving largely independently. This encourages parallel exploration of different regions of the search space and increases the likelihood of discovering multiple local optima.

To further enhance diversity, islands are **specialized** by assigning each island a distinct crossover operator probability distribution. As a result, different islands favor different recombination mechanisms, leading to complementary search behaviors. Periodic **ring migration** is used to exchange individuals between neighboring islands, allowing high-quality solutions to propagate while maintaining sufficient isolation to preserve diversity. The **best solutions** of each island are **migrating** and **replacing** the **worst solutions** of the next island.

Within each island, **fitness sharing** is used as an additional diversity promotion mechanism. Fitness sharing penalizes individuals that are too similar to others, thereby discouraging overcrowding in a single region of the

search space. The sharing mechanism is **adaptively activated** based on the normalized Hamming distance of the island population. When the diversity falls below a predefined threshold $D_H < \tau$, fitness sharing is enabled. Otherwise, it is disabled to avoid unnecessary distortion of the fitness landscape.

These diversity promotion mechanisms improve the performance of the evolutionary algorithm by balancing exploration and exploitation. The island model enables **global exploration** across multiple basins of attraction, while adaptive fitness sharing preserves **local diversity** within islands. The approach introduces several hyperparameters, including the number of islands, migration interval, fitness sharing threshold, and crossover specialization strength. These parameters were selected empirically.

3.11 Stopping criterion

A stopping criterion is required to balance solution quality against computational cost. In time-constrained settings, it can be advantageous to terminate the algorithm once the marginal improvement per unit of time becomes sufficiently small. Conversely, when a fixed computational budget is available, prematurely stopping the algorithm may lead to underutilization of available resources and suboptimal solutions.

In more realistic scenarios—such as hyperparameter tuning, where multiple algorithm runs are required—we employed alternative stopping criteria during development to speed up the process. In particular, a **patience-based criterion** was used, terminating the run after a predefined number of iterations without improvement in the best objective value. Other criteria were considered, including a fixed maximum number of iterations and termination based on **global diversity collapse**. However, a **fixed iteration limit** was deemed unreliable, as the computational cost per iteration varies with algorithmic parameters and problem size.

For the final benchmark experiments, we chose not to apply an early stopping criterion. Instead, the algorithm was allowed to run for the full time budget of five minutes in all cases, ensuring full utilization of the available computational resources.

We also considered a **restart-based mechanism** in which the population is reinitialized once stagnation is detected. Although such restarts can increase exploration, empirical observations indicated that **continued optimization** without restarting occasionally led to further improvements after periods of stagnation. Given the associated time cost of reinitialization, restart mechanisms were therefore excluded from the final algorithm.

3.12 Parameter selection

For parameters that are not automatically adjusted through adaptive or self-adaptive mechanisms, we performed a **systematic hyperparameter search** using the optimization framework **Optuna**. Optuna employs a model-based optimization strategy to construct an internal surrogate model of promising hyperparameter configurations and to efficiently explore the parameter space.

The hyperparameter optimization was conducted over multiple problem instances with varying tour sizes. For each configuration, performance was evaluated using the final objective value obtained within a fixed computational budget. The best-performing configurations identified by Optuna were subsequently validated on unseen instances and further refined through limited manual tuning. This manual analysis provided additional insight into parameter sensitivity and interactions.

4 Numerical experiments

(target: 1.5 pages)

4.1 Metadata

The hyperparameters of the algorithm used in the numerical experiments are listed in Table 1.

All experiments were conducted on a machine equipped with an AMD Ryzen 7 5800H processor featuring **8 physical cores** and 16 hardware threads, with a **maximum boost clock** of up to **4.47 GHz**. The system supports AVX2 instructions, and is equipped with 13 GB of main memory. All algorithms were implemented and executed using Python 3.11.

4.2 tour50.csv

The algorithm was run on benchmark tour 50 with a time limit of 5 minutes per run. The best solution found is:

- **Tour length:** 13844.338305
- **City sequence:**

0, 36, 24, 3, 1, 40, 11, 10, 38, 8, 22, 6, 19, 23, 49, 2, 32, 15, 43, 45,
26, 20, 47, 44, 17, 12, 29, 13, 31, 27, 25, 16, 42, 18, 21, 48, 5, 35, 41,
30, 7, 9, 14, 37, 33, 34, 28, 39, 4, 46

Parameter	
Population size	270
Initialization noise scale	0.01
k-tournament	2
Mutation base	0.1
Mutation high	0.2
Mutation patience	263
Swap ratio	0.55
Inversion ratio	0.35
Scramble ratio	0.10
Crossover min	0.45
Crossover max	0.65
OX ratio	0.15
EPX ratio	0.65
ERX ratio	0.20
Elitism ratio	0.04
LSO max	0.25
LSO min	0.05
Max improvement two-opt	20
Max improvement three-opt	2
Max improvement or-opt	12
Two-opt ratio	0.50
Three-opt ratio	0.05
Or-opt ratio	0.45
Max segment length or-opt	3
Crossover diversity	0.289
Crossover specialization	4.0
Fitness sharing threshold	0.1258
Sigma	0.815
Alpha	2.69
Number of islands	8
Migration interval	300
Patience	500

Table 1: Hyperparameters used in the evolutionary algorithm for different TSP instances

Although the algorithm eventually finds the global optimum (tour length 13844.34) after 300 iterations, the mean population fitness remains significantly higher than the best solution throughout the run (Figure 3). This indicates that a substantial portion of the population is still exploring suboptimal regions. Consequently, stopping criteria based on population diversity are ineffective for detecting convergence to the optimum in the current design. Therefore in this implementation, the algorithm is stopped after an arbitrary tuned **patience** parameter of **500 iterations**.

Adjusting the (λ, μ) ratio to increase offspring relative to surviving individuals may also accelerate population convergence, allowing diversity-based criteria to become meaningful. This could potentially reduce the time wasted on exploring further when the minimum is already found.

The algorithm was run **500** times on the **50-city tour** problem, recording both the **final best fitness** and the **final mean fitness** of the population in each run. The resulting histograms are shown in Figure 4.

The algorithm consistently finds near-optimal solutions, with only eight runs (1.6%) producing suboptimal results, indicating high reliability. The histogram of the final best fitnesses is sharply peaked around the global minimum, showing low variance. In contrast, the histogram of the final mean fitnesses exhibits greater variability due to the stochastic nature of the algorithm and the fact that the population means do not always converge toward the optimal solution.

The first and second moments of the data are as follows:

$$\begin{aligned}\mu_{\text{best}} &= 13\,845.002, & \sigma_{\text{best}} &= 5.215 \\ \mu_{\text{mean}} &= 15\,232.779, & \sigma_{\text{mean}} &= 396.524\end{aligned}$$

In a previous prototype design that considered $(\lambda + \mu)$, the means would collapse toward the best objective, resulting in a histogram with much lower variance.

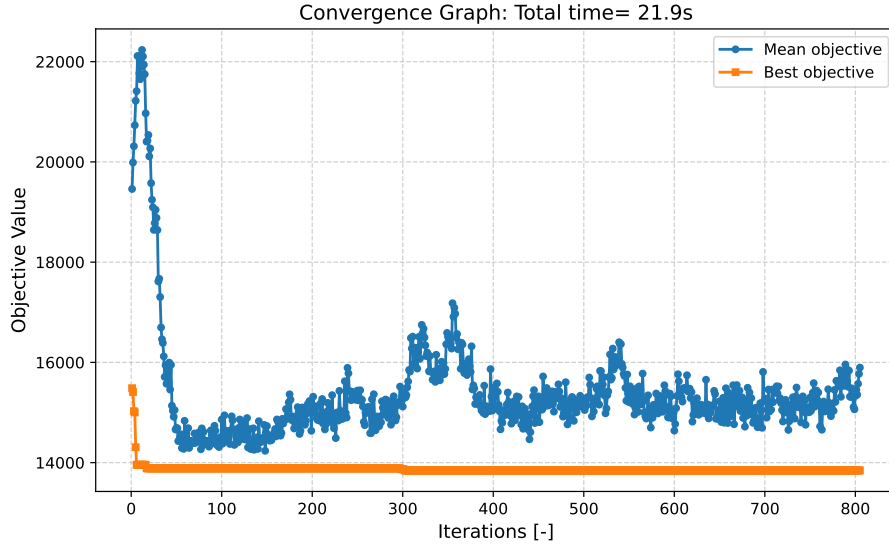


Figure 3: Convergence graph of tour 50 with a patience of 500 iterations

Overall, these results demonstrate that the algorithm is highly consistent in finding optimal tours, while variability in the population mean reflects its stochastic behavior.

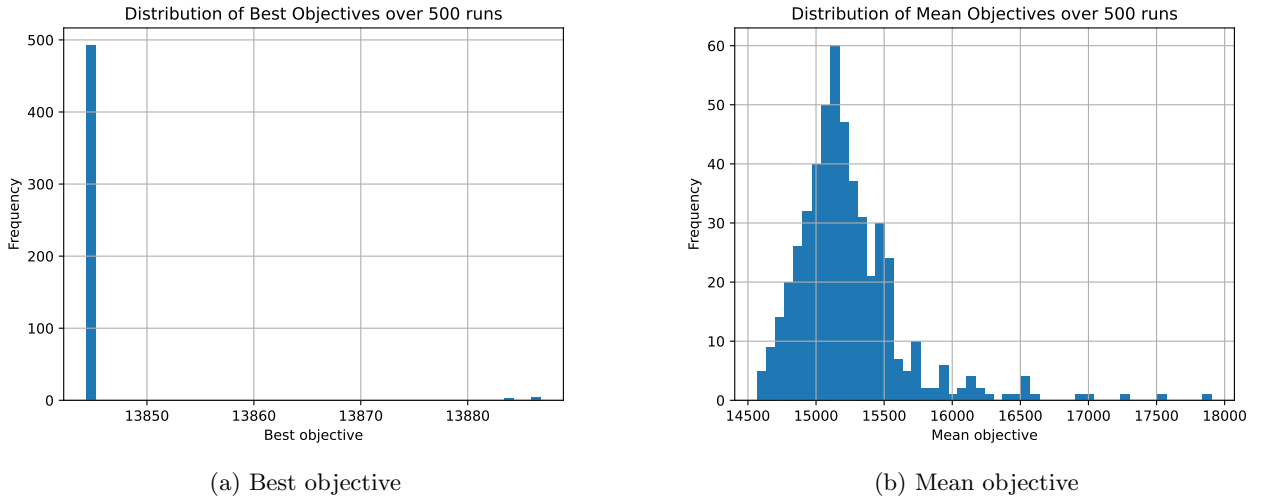


Figure 4: Histograms of final objective values for tour 50 over 500 runs.

4.3 tour250.csv

The algorithm was executed on the **250-city tour** benchmark using the imposed **5-minute time limit**. A typical convergence graph, showing both the best and mean objective values as a function of time, is presented in Figure 5.

The best tour length found during the run was **76 280.169**, while the final mean objective value of the population was **90 814.560**. As shown in the convergence graph, the algorithm achieves a rapid decrease in the objective value during the initial phase, indicating effective early exploration. However, after this phase, the convergence stagnates and only marginal improvements are observed.

Despite this stagnation, the population remains diverse throughout the run, which suggests that the algorithm prioritizes exploration over aggressive exploitation. While this helps prevent premature convergence, it also limits the algorithm's ability to refine solutions sufficiently to reach the global optimum within the available time budget.

Overall, the algorithm demonstrates reasonable performance in terms of speed of initial convergence and population diversity, but it is **unable to reach the global minimum** for this problem size within the time limit.

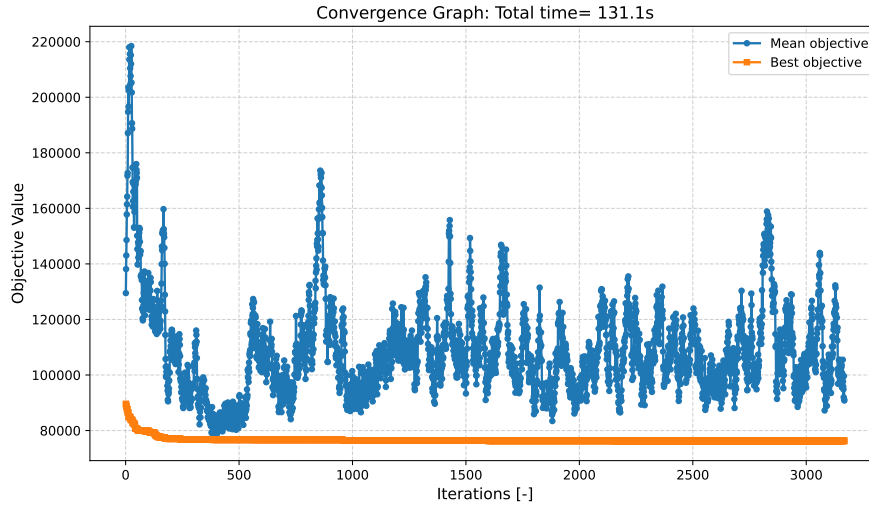


Figure 5: Convergence graph for the 250-city tour (patience = 1000 iterations)

4.4 tour1000.csv

The algorithm was also evaluated on the significantly larger **1000-city tour** benchmark under the same time constraints. The convergence behavior is illustrated in Figure 6.

The best objective value obtained was **60 466.194**, while the final mean objective value of the population became **infinite**. This occurs because some individuals in the population contain infeasible tours with infinite length, which heavily affects the population mean. As a result, the mean objective value is no longer a reliable performance indicator for this problem size.

The convergence graph shows a strong initial descent, indicating that the algorithm is capable of quickly identifying high-quality solutions. After this initial phase, the optimization process stagnates, with only limited improvements over time. Notably, the algorithm is able to find a competitive solution relatively early within the time limit, after which progress slows considerably.

Due to the problem scale and the presence of infeasible individuals, the algorithm struggles to further refine solutions and is again **unable to reach the global minimum**. While the algorithm scales reasonably in terms of memory and early convergence speed, its performance is limited by stagnation and the difficulty of maintaining feasibility and exploitation in very large search spaces.

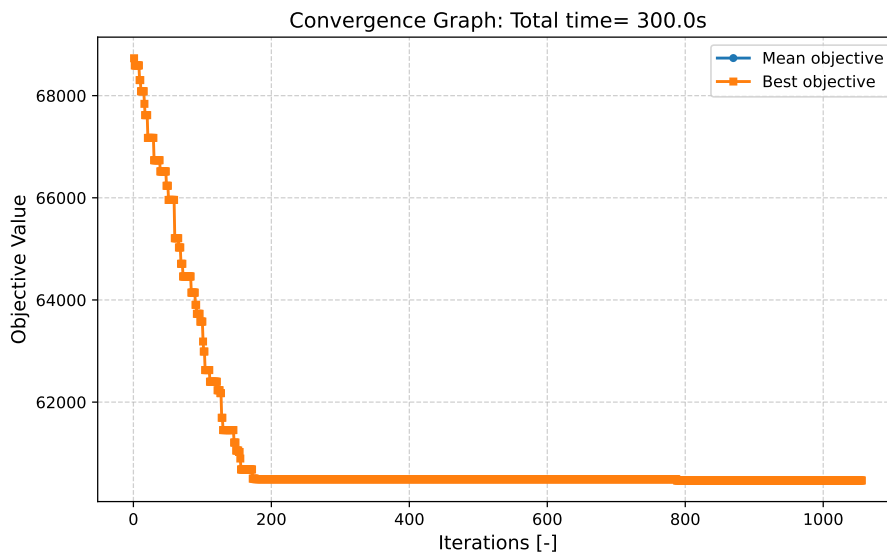


Figure 6: Convergence graph for the 1000-city tour (patience = 500 iterations)

5 Critical reflection

(target: 1.5 page)

5.1 Strengths and weaknesses

The three main strengths of evolutionary algorithms in my experience are:

1. Evolutionary algorithms are **general-purpose** methods for **global optimization**, as they make few assumptions on the objective function and can be applied to black-box, non-convex, and discontinuous problems.
2. They offer explicit **control** over the **exploration-exploitation trade-off** through the design and tuning of operators such as mutation, crossover, selection pressure, and local search.
3. They are well suited as a **first approach** to obtain good solutions, which can later be refined using problem-specific heuristics or local optimization methods.

The three main weaknesses of evolutionary algorithms in my experience are:

1. Evolutionary algorithms rarely outperform highly specialized algorithms that exploit strong problem-specific structure.
2. They are **difficult to tune**: poor hyperparameter choices can lead to excessive and wasteful function evaluations, as interactions between parameters are often complex and problem-dependent.
3. Designing effective mutation and crossover operators is non-trivial, since they are often highly problem-specific and strongly influence performance.

The main **lessons learned** from this project are, first of all, **how to design and implement an evolutionary algorithm**, and how to **balance exploration and exploitation**. In particular, the importance of exploitation became clear: adding local search operators significantly improved the performance of the algorithm. I believe evolutionary algorithms are **appropriate** for the **Traveling Salesman Problem** when the goal is to obtain good-quality solutions within reasonable time. However, they are unlikely to outperform highly specialized heuristics such as the **Lin-Kernighan-Helsgaun (LKH)** method, which exploit strong problem-specific structure. One surprising aspect was how often evolutionary algorithms **behave** in a **counterintuitive** way. Due to the large number of hyperparameters and their complex interactions, small parameter changes can lead to unexpected effects. For example, in one tuning setup, decreasing the probability of the local search operator caused the population to collapse, while increasing it too much led to the same effect. This illustrates how **sensitive** evolutionary algorithms can be to **parameter tuning**. Overall, I learned how to design evolutionary algorithms for global optimization problems, how to incorporate exploitation mechanisms effectively, and how systematic hyperparameter tuning can strongly influence performance. In addition, I learned how **generative AI** can be used as a practical support tool to enhance algorithm development, for instance by aiding in implementation, debugging, and systematic exploration of algorithmic design and parameter choices.

5.2 GenAI peer reviewing critical reflection

The **AI-generated peer review** was **fast, well-structured, and concise**, making it very easy to read. Its feedback tended to be more obvious and less creative, focusing on general points.

The **human review**, on the other hand, was **longer and more creative**, often offering deeper or less obvious insights into the algorithm. However, it was slightly **harder to read** due to its length and occasional grammatical mistakes. Both reviews identified issues that the other did not, showing that **AI can complement human reviewers** rather than replace them.

Writing my own peer review was particularly helpful in deepening my understanding of the other group's algorithm. By analyzing their design choices, assumptions, and experimental results, I engaged more critically than I would have by simply reading the AI's review.

At the same time, relying too heavily on AI can be risky. Overtrusting generative AI may encourage laziness, reduce critical thinking, and expose one to mistakes caused by AI hallucinations. Therefore, while AI is a valuable tool for rapid, concise feedback and initial guidance, it should not replace human judgment in evaluating scientific work.

In conclusion, generative AI can serve as a **useful support tool** in peer review, providing **quick, structured, and easy-to-read feedback**. Human reviewers, however, bring **context-aware insights, creativity, and deeper understanding**, which are essential for thorough evaluation. Personally, I would prefer to receive **human reviews** for **comprehensive feedback** while using **AI reviews** as a **complementary tool** for **rapid checks** and **preliminary guidance**.

5.3 GenAI critical reflection

Generative AI **helped** me during the project in several ways. It assisted in understanding and finding good recombination and local search operators, and it was useful for **translating my algorithmic ideas into working code**. GenAI **allowed** me to **focus more on the design** and structure of the algorithm rather than spending excessive time on implementation details.

However, blindly copying code from GenAI can lead to problems. For example, when converting some Python functions into Numba, GenAI introduced a subtle bug that added a "-1" in my tours. Moreover, over-reliance on AI can make one lazy and reduce the motivation to fully understand the underlying concepts, which can hinder learning and deep comprehension.

The activities for which I did observe sufficient added value:

1. **Understanding** and **implementing** specific algorithmic operators (e.g., recombination and local search).
2. Quickly **prototyping** code for algorithmic ideas.
3. Allocating more time to the design and evaluation of the evolutionary algorithm rather than low-level coding details.

For which activities I did not observe any or sufficient added value:

1. Debugging subtle algorithmic or implementation errors. The AI sometimes introduced bugs.
2. **Critical thinking** about algorithm design choices. The AI almost always goes with your idea where it rarely is able to critically think through your ideas.
3. Developing a deep understanding of the behaviour of the evolutionary algorithm—AI can provide code, but not intuition or insight.

My **project solution** would have been **substantially different without** relying on **GenAI**, primarily in terms of **complexity**. Without AI assistance, implementing all algorithmic operators and coding the evolutionary algorithm would have taken significantly more time, likely making a more complex solution less feasible.

Using GenAI allowed me to gain considerable time in implementing operators and prototyping ideas, which outweighed the time lost debugging occasional errors introduced by AI. Additionally, using LLMs to quickly look up information acted like a **fast search engine**, helping me find relevant sources and references efficiently.

6 Other comments

(does not count toward page limit)

To efficiently implement and tune the algorithm, a time profiling tool, namely **cProfile**, was used. This enabled a detailed analysis of the execution time spent in individual components of the algorithm and provided insight into the most computationally expensive subroutines. As a result, performance bottlenecks and unnecessary computations could be identified and addressed in a targeted manner.

In addition, the Hamming diversity of each island was logged during execution, providing immediate feedback on the evolution of population diversity throughout the optimization process. This diagnostic information proved valuable during development and tuning, as it facilitated a better understanding of the algorithm's dynamics and supported informed design decisions.

7 Generative AI compliance form

(does not count toward page limit)

7.1 Exploring initialization options

Generative AI model: ChatGPT v4

Motivation: The goal was to quickly get an overview of initialization techniques for the Traveling Salesman Problem.

Methodology: *Prompt 1:* Can you help me design an evolutionary algorithm to tackle the traveling salesman problem. I need a good initialization heuristic which takes into account infinite cost between some cities (no connection considered infinite wait) What are my options?

Prompt 2: The problem will be maximum 1000 nodes and time is limited to 5 min. So therefore speed is important and runtime complexity as well, For each initialization heuristic can you list the runtime complexity for it?

Prompt 3: The graphs can be expected to be dense! So m will be $O(n^2)$

Postprocessing: /

Reflection: The LLM gave me an extensive overview of possible initializations.

7.2 Generating code skeleton

Generative AI model: ChatGPT v4

Motivation: I wanted to obtain a good well-thought code structure which was robust and clear without compromising on computational speed. Instead of starting to implement my algorithm without thinking it through I wanted to verify that indeed my starting skeleton is modular, easily extendable while still being fast.

Methodology: *Prompt 1:* This is the template code i have received and which should be respected. I need to implement genetic algorithm. I will have a lot of hyperparameters to tune and i would like to have a library from pip doing this for me to generate reports etc. Also i want to use numba to speed up my calculations Can you give me an idea of how to structure this code? Where i can put my functions of the algorithm where i should have my hyperparameters And how to use numba for it efficiently?

Prompt 2: make a conda environment for this please

Postprocessing: The generated code was used verbatim.

Reflection: The LLM generated a code skeleton and filled components in with basic operators. The skeleton was modular, easily extendable and was able to contain numba functions. The algorithm worked and I was able to start filling in these components with more advanced operators to my liking.

7.3 Generating initialization methods

Generative AI model: ChatGPT v4

Motivation: Quickly implement greedy, DFS, BFS algorithm as initialization.

Methodology: *Prompt 1:* Help me generate these initialization functions for traveling salesman problem Main initialization method, is taking weighted initialization from the different initialization functions

Postprocessing: After checking if the algorithms were implemented correctly, the individual functions were able to be used verbatim. However, I implemented the weighted initialization myself, since I did not like the coding style of the generated output of the LLM.

Reflection: The initialization functions worked, and produced valid tours.

7.4 Change indentation convention

Generative AI model: ChatGPT v4

Motivation: The generated code had the convention of using spaces instead of tabs (which I use). This led to indentation errors which I wanted to get fixed.

Methodology: *Prompt 1:* Can you make the indentation convention use tabs instead of spaces? This is annoying
Prompt 2: But the rest of the code still uses spaces, make them use tabs as well

Postprocessing: The output was used verbatim.

Reflection: Problem solved.

7.5 Exploring alternative options for the selection step

Generative AI model: ChatGPT v4

Motivation: The goal was to quickly get an overview of existing parent selection techniques before deciding which option to implement. I wanted to explore the possible alternatives and understand their main benefits and drawbacks.

Methodology: *Prompt 1:* Are there other interesting ways to do parent selection?

Postprocessing: I reviewed the suggestions and compared them against my project constraints. I ultimately decided to stick with k-tournament selection because it is easy to implement and works well for small population sizes.

Reflection: The generated output provided a concise overview of the pros and cons of eight selection methods. This helped me quickly consider alternative approaches. While I did not directly use all methods, it was useful for understanding the design space and confirming that my chosen method was appropriate.

7.6 Python script to plot convergence

Generative AI model: ChatGPT v4

Motivation: Quickly get some feedback on the convergence of the algorithm. In order to guide the design of the method.

Methodology: *Prompt 1:* This is my .csv file Make a python script to read it and plot the best and mean value over iterations please

Postprocessing: The code was used verbatim.

Reflection: It did indeed give insights into the algorithm's convergence.

7.7 Feedback on variation operators

Generative AI model: ChatGPT v4

Motivation: Get some quick feedback on a thought related to swap mutation and ordered crossover.

Methodology: *Prompt 1:* these variation operators should the amount of swaps and length of order switch be related to the problem size?

Postprocessing: /

Reflection: The AI gave a satisfactory explanation which was consistent with my thought that it should be.

7.8 Pluggable crossover method

Generative AI model: ChatGPT v4

Motivation:

Methodology: *Prompt 1:* Rewrite this code such that the crossover method is given in `self.crossover(parent1, parent2, method)` And `num_ordered = int(self.population*self.num_ordered)` for `method, i` in `zip(methods, range(elitism, num_individuals))`

Postprocessing: The code worked as intended.

Reflection: The AI implemented the method correctly. However, later in the design this implementation was removed due to additional python overhead.

7.9 Numba implementation of mutation operators

Generative AI model: ChatGPT v4

Motivation: Convert existing mutation operators into numba functions to accelerate the algorithm.

Methodology: *Prompt 1:* Inserted existing code with `@njit` implement swap mutation and inversion mutation

Postprocessing: The output was used verbatim

Reflection: The methods worked perfectly and as intended.

7.10 Pitch initialization idea related to infinities

Generative AI model: ChatGPT v4

Motivation: I quickly pitched an idea about preferring solutions with less infinities than more. Since I was struggling with getting feasible solutions for matrices with large amount of infinities.

Methodology: *Prompt 1:* Okay i have a problem at problem size of 750 cities since here infinite edges are added The problem is that my random initialization doesn't work out. Should i in this case count the amount of infinities to prefer cities with less infinities then cities with more?

Postprocessing: /

Reflection: The model suggested not to count infinities since this would make it less likely to pick bad cities early, but the permutation still will almost always form illegal transitions. Therefore it suggested to utilize constructive initialization strategies. This is where I decided to leave out the completely random initialization, especially for problems with large amount of cities.

7.11 Print output summary

Generative AI model: ChatGPT v4

Motivation: Quickly print some usefull information about the iterations after running the method.

Methodology: *Prompt 1:* print some info on the data: best value, max iterations, mean values ...

Postprocessing: The output was used verbatim.

Reflection: The LLM generated a nice summary of the algorithms data.

7.12 Crossover strategies for dense graphs

Generative AI model: ChatGPT v4

Motivation: Get an overview of crossover strategies that might be beneficial for dense graphs.

Methodology: *Prompt 1:* Get an overview of crossover strategies that might be beneficial for dense graphs.

Postprocessing: /

Reflection: The GenAI produced a comprehensive summary of possible crossover strategies for dense TSP.

7.13 Prototyping local search operator

Generative AI model: ChatGPT v4

Motivation: Quickly implement a basic local search operator to evaluate gain in performance.

Methodology: *Prompt 1:* I want to add a local search operator for my TSP Context: benchmarks on small problems N =50 to big problems N=1000

Prompt 2: Why is best_delta not used?

Prompt 3: is this implementation correct.

Prompt 4: what would be a good LSO probability?

Prompt 5: IndexError: index 50 is out of bounds for axis 1 with size 50

Postprocessing: The implementation of the two-opt LSO was not perfect. The AI had made an array indexing error. So a little tweaking was necessary. The problem was that in this chat with the LLM, it assumed my cities were 1-based instead of 0-based.

Reflection: The GenAI produced a comprehensive summary of possible crossover strategies for dense TSP.

7.14 Feedback on adaptive continuous mutation rate

Generative AI model: ChatGPT v4

Motivation: Get feedback on the idea of continuously varying the mutation rate adaptively.

Methodology: *Prompt 1:* # Adaptive mutation if no_improvement % self.mutation_patience == 0: self.mutation_rate += self.mutation_increase*mutation_rate_original print(f"Mutation rate increased to: self.mutation_rate")
Is this a good idea or should it be more like discrete values? High mutation rate and low mutation rate

Postprocessing: /

Reflection: The AI reviewed the idea and thought it could be good. However, due to limited gain in performance i left it out of the algorithm and stuck with discrete mutation rate variation for simplicity.

7.15 Converting existing fuctions into Numba routines

Generative AI model: ChatGPT v4

Motivation: To convert existing Python functions into Numba-accelerated routines to significantly reduce the runtime per iteration in the evolutionary algorithm.

Methodology: *Prompt 1:* Can you convert these two methods into a numba version?

Postprocessing: Some issues arose because Numba does not support class instances directly. I manually moved the functions outside of the class and ensured that no class objects were passed as arguments. After these adjustments, the functions compiled and executed correctly with Numba.

Reflection: The implementation of the Numba functions was correct and worked well. I provided the LLM with only the two functions rather than the entire code to avoid overloading it. Had I prompted with the whole code, it is likely that the model could have produced a directly working implementation as well.

7.16 Implementation of local search operator

Generative AI model: ChatGPT v4

Motivation: To implement an asymmetric local search operator: Or-opt.

Methodology: *Prompt 1:* can you implement Or-opt for me for the TSP problem?

Prompt 2: This is three opt implementation in numba Could you do Or-opt in this framework as well?

Postprocessing: The routine was implemented correctly. I modified the function arguments to make parameters such as the maximum number of improvements and the maximum segment length adjustable.

Reflection: The output produced by the LLM was correct and required minimal manual adjustments.

7.17 Island specialization and diversity across islands

Generative AI model: ChatGPT v4

Motivation: I wanted to explore options for defining mathematical functions to tune the diversity across islands and to allow islands to specialize in specific crossover types, while keeping both diversity and specializability tunable via hyperparameters.

Methodology: *Prompt 1:* I have a genetic algorithm trying to tackle TSP I use island model have mixed probabilities for three types of crossover, three types of mutation and three types of local search operators I would like to have some hyperparameters tuning the specializability and diversity of each island Then a function will apply this to each island and give them different ratios for the types of crossover, mutation and local search ops I would like to have something like: `crossover_speciality` -> If high it pushes crossover types to become specialized > 90% ratio in one of the three crossover types `crossover_diversity` -> if high equally spreads crossover types through all islands ... How could i design such a function? Given `crossover_diversity` and `crossover_speciality` and a base distribution which i know is already good?

Postprocessing: The model provided a clear conceptual approach for designing such functions. Since the implementation was relatively straightforward, I coded the functions myself and adapted them to fit my existing code structure.

Reflection: The generative AI output was useful in suggesting a structured way to implement specialization and diversity across islands for crossover operators. It helped me quickly conceptualize a solution controlled by two hyperparameters, even though the actual coding was done manually.

7.18 Time profiling of algorithm

Generative AI model: ChatGPT v4

Motivation: I needed a way to identify where my algorithm was spending most of its time, in order to pinpoint bottlenecks—whether implementation-related or algorithmic.

Methodology: *Prompt 1:* I would like to get a report on how long some important functions take in my genetic algorithm in python. So that i have an idea which subroutines are the bottleneck and which i need to optimize my code for. Could you help me get started on this idea. Should i implement a time profiler myself or are there packages available suiting my purpose?

Postprocessing: The model suggested several options, and I chose the simplest built-in solution: `cProfile`. I implemented a basic profiler script to measure the execution time of each function. This was especially useful to identify code that had not yet been accelerated with Numba. Later, it also helped decide whether optimization efforts should focus on local search operators or recombination routines.

Reflection: The output was very useful. It provided a quick and easy way to gain insights into function-level execution times, guiding both code optimization and algorithmic improvements.

7.19 Implementation of segment swap LSO

Generative AI model: Gemini 1.5 Flash

Motivation: I wanted to quickly test a new local search operator, the segment swap LSO.

Methodology: *Prompt 1:* Can you implement another LSO, Namely segment swaps please?

Postprocessing: The generated code contained a subtle bug related to array index management: some '-1' entries remained in the tours. The algorithm initialized tours with '-1' and then filled in the remaining positions, but an error in the loops left some positions unfilled. I manually identified and corrected the issue.

Reflection: The model was not very effective for coding tasks. While Gemini may be strong at mathematical reasoning, it did not produce reliable code in this case, and manual intervention was required.

7.20 Update optuna optimization script

Generative AI model: ChatGPT v4

Motivation: I wanted to quickly update my optuna optimization script after some new hyperparameters were introduced into the design.

Methodology: *Prompt 1: "Inserted code"* Can you update my optuna optimization script to include my current parameters please?

Postprocessing: The generated code contained was used verbatim.

Reflection: It worked well, saved me manual typing of some lines.